1 Evaluation Methodology

This chapter details the methodology used to evaluate the **Agentic Project Management (APM)** framework. To assess the framework's effectiveness against current industry standards, we conducted a comparative case study involving human participants.

The evaluation was designed to measure two distinct dimensions of efficiency:

- **Economic Efficiency** (monetary and computational cost)
- **Process Efficiency** (human cognitive load and interaction burden)

To accurately capture these distinct metrics, the study was divided into two complementary sub-studies, leveraging the unique data-collection capabilities of two different IDE platforms (Cursor and VSCode with GitHub Copilot).

1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM (i.e., the Setup and Task Loop phases) yields a tangible return on investment compared to single or double-agent workflows.

We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):** Does the multi-agent structure of APM significantly increase the token consumption and financial cost compared to single-agent or double-agent workflows?
- **RQ2 (Process Efficiency):** Does the structured coordination of APM reduce the "human burden" during execution? Specifically, does it reduce the need for user interventions, corrections, and clarifications? Are the agents able to effectively coordinate and complete the task without the need for constant human oversight?
- **RQ3 (Functional Efficacy):** Can APM successfully deliver complex, non-trivial features in scenarios where unstructured agentic workflows typically encounter "Agentic Handoff Failures"?

1.2 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we utilized an existing open-source project, the `eclass-mcp-server`, a Python-based Model Context Protocol (MCP) server designed to interface with the `openeclass` platform. Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments.

1.2.1 The Task:

Participants were tasked with implementing new functionality for the MCP server. The task evolved through iteration: an initial prototype task (adding a new feature to the MCP server) proved insufficient for differentiating multi-agent from single-agent approaches; the latter handled it without difficulty. The final task specification adds a **C replica** requirement, demanding both the Python feature implementation and a native C implementation of the MCP server.[1]

The C replica requirement was intended as a **context window stress test**. By forcing agents to manage complex, cross-paradigm context and multiple constraints, this task increases cognitive load and the risk of context failures for single-agent approaches. This highlights the advantages of multi-agent coordination, where structured separation of concerns and workload distribution better manages such complexity.

Key Technical Challenges:

1. **Dynamic Data Source:** The required data for implementing the new feature is not available in static HTML for parsing. Agents must research the legacy codebase to discover a specific AJAX endpoint and its behavior.
    - **Header Requirements:** The endpoint requires specific headers (e.g., `X-Requested-With: XMLHttpRequest`) to return valid JSON.
2. **Nested Parsing:** The server returns JSON where the payload is an array of raw HTML strings, requiring a second layer of parsing to extract the data.
3. **Client-Side Filtering:** The legacy API does not support date filtering. The agent must implement robust client-side logic to parse and filter unique locale-formatted dates.
4. **C Implementation:** Agents must implement a native C replica of the MCP server, requiring implementation of MCP's JSON-RPC 2.0 protocol from scratch in native C (no external libraries), and strict adherence to external library usage for HTTP requests and HTML parsing in eclass-specific functionality.
5. **Memory Safety:** The C implementation must be memory-safe, passing Valgrind checks without leaks or errors.

Deliverables:

1. Core server logic in modular Python files (existing and new functionality).
2. A native C implementation of the MCP server with equivalent (existing and new) functionality.
3. A working build system (Makefile) for the C replica.

Submissions are evaluated using an automated black-box test suite.[2]

[1] Task specification and grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task

[2] Grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task/tree/main/grader

1.2.2 Repository Architecture and Data Management

To ensure participant privacy and robust data security, the case study employs a two-repository system that clearly separates public resources from sensitive research data. This architecture balances research transparency with participant confidentiality: it ensures that dataset components necessary for peer review and reproducibility are available through controlled channels, while personally identifiable material and unsanitizable transcripts are never made public.

The **public repository** (`bsc-apm-case-study-task`) is openly accessible and includes:

- Task specification documents (Product Requirements Document, Default Prompt)
- Grading infrastructure and automated test suite
- Participant guides and setup instructions

The **restricted-access repository** (`bsc-apm-case-study-data`) contains all submitted participant data and sensitive interaction records:

- Code patches and implementation artifacts
- Sanitized AI assistant chat transcripts – identity-revealing information (such as file paths, tool calls, timestamps, and user IDs) is systematically filtered out. However, it is not possible to fully sanitize unstructured user-input text, which may inadvertently reveal identities. Therefore, raw interaction logs are never published, and even sanitized transcripts remain in the restricted repository.
- Session metadata and metrics (fully anonymized wherever possible)

Access to the restricted-access repository is available only upon request, following the signing of formal data usage agreements and compliance with data access terms, which are subject to final approval prior to publication.[3] This controlled access mechanism enables peer review and reproducibility while maintaining participant privacy protections.

1.2.3 Sanitization and PII Handling in Study B

GitHub Copilot chat exports contain sensitive information (file paths, command histories, timestamps). To protect participant privacy while enabling reproducibility, exports are sanitized before storage using reversible encryption.

Sanitization Method:

Fields containing personally identifiable information are encrypted using an HMAC-SHA256 XOR cipher:

- **Keystream Generation:** HMAC-SHA256 is applied to a combination of a shared secret and per-field random salt, generating a keystream matching the field length.
- **XOR Encryption:** The plaintext field is XORed against the keystream, producing ciphertext of identical length.

[3] Data access terms and consent forms are currently under review and will be finalized prior to publication.

- **Encoding:** Encrypted fields are stored as `ENC:<salt_b64>:<ciphertext_b64>`, enabling deterministic decryption.

Coverage: File paths, tool arguments, command histories, and text content containing paths are encrypted. Session IDs (computed as SHA256 hashes of request IDs) remain in plaintext for session tracking.
Integrity: Decryption using the shared secret produces data bitwise-identical to the original. Metrics computed from sanitized exports are identical to those from unsanitized data, ensuring analysis validity.

1.3 The Two Sub-Studies:

To provide a holistic evaluation, we split the experiment into two sub-studies. This design choice was driven by the need to isolate specific variables and leverage the distinct logging capabilities of the available tools.

Table 1: Comparison of Sub-Study Designs

Feature	**Sub-Study A (Cursor)**	**Sub-Study B (GitHub Copilot)**
Primary Focus	Quantitative Economic Analysis	Qualitative Process Analysis
Target Audience	Pro Users (Controlled Group)	University Students
Primary Metric	Tokens (I/O) & Cost ($)	Interaction Patterns & User Burden
Data Source	Cursor Usage Statistics	Exported GitHub Copilot JSON Chat Logs
AI Model Strategy	**Standardized:** Cursor Composer 1 (All chats)	**Hybrid:** Premium (Setup and Manager) + Budget (Implementation)

1.3.1 Sub-Study A: Quantitative Economic Analysis (Cursor)

This study focused on **RQ1**. It utilized a smaller, controlled group of participants with access to Cursor's premium AI features.

- **Rationale:** Cursor provides precise, session-based dashboards for Token Input/Output/Cache and Cost per request.
- **Model Standardization:** To ensure rigorous comparison, all participants were strictly required to use **Cursor's Composer 1** model for the entire duration of the study. This eliminates model variance as a confounding variable for cost.

1.3.2 Sub-Study B: Qualitative Process Analysis (GitHub Copilot)

This study focused on **RQ2**. It utilized a larger cohort of university students with access to GitHub Copilot's premium AI features (via the GitHub Education Pack).

- **Rationale:** GitHub Copilot provides detailed JSON chat exports that include metadata on tool calls, timestamps, and speaker roles. This granular data allows for programmatic analysis of interaction patterns that is not possible with Cursor's summary dashboard.
- **Hybrid Model Strategy:** To mimic real-world usage constraints (request quotas), participants in the APM group followed a hybrid strategy: using a Premium Model (Claude Sonnet 4) for high-level planning (Setup/Manager agents) and a Budget Model (with a 0x request multiplier) for execution (Implementation agents).

1.4 Frameworks Under Evaluation

Three distinct workflow configurations were evaluated across both sub-studies:

1. **Agentic Project Management (APM):** The proposed framework. It enforces a strict separation of concerns:
   - *Setup Agent:* Project discovery and planning.
   - *Manager Agent:* Coordination, task assignment, and review.
   - *Implementation Agent:* Execution and memory logging.
2. **GitHub Spec-kit:** A structured, "constitutional" baseline. It utilizes a single-agent workflow guided by rigid templates and three sequential commands (`specify`, `plan`, `tasks`).
3. **Plan Mode (Baseline):** The industry-standard "First plan, then execute" feature native to the IDE (Cursor's Plan Mode or GitHub Copilot's Plan Agent). This represents the current state-of-the-art for AI-powered planning workflows.

1.5 Metrics and Data Collection

1.5.1 Economic Metrics (Study A)

Collected via Cursor's usage statistics of each participant's session to address **RQ1**:

- **Total Input Tokens:** The volume of context processed by the models during the session.
- **Total Output Tokens:** The volume of code and text generated during the session.
- **Total Cached Tokens:** The volume of tokens served from Cursor's cache, reducing overall cost.
- **Total Cost ($):** The financial cost of the session based on Cursor's API pricing.
- **Time to Completion:** The total elapsed time from task initiation to final submission, measured in minutes.

1.5.2 Process Efficiency Metrics (Study B)

Derived from parsing sanitized GitHub Copilot JSON chat transcripts to address **RQ2**. Metrics are organized into three tiers of increasing interpretive complexity:
Session Metadata:

- **Session ID:** SHA256 hash (first 16 chars) of the chat request ID.
- **Model ID:** The specific language model used (e.g., `copilot/claude-opus-4.5`).

Core Metrics (Tier 1) — Direct Measures of Autonomy and Effectiveness:

- **Total Requests:** Number of user prompts.
- **Total Tool Invocations:** Sum of all tool calls across all requests.

- **User Edit Count:** Times the user modified an auto-generated terminal command (proxy for correction burden).
- **Terminal Invocations:** Total number of terminal command tool calls.
- **User Edit Rate:** Ratio of edited to total terminal invocations; lower indicates fewer corrections needed.
- **Tool Success Count:** Number of successfully completed tool invocations.
- **Tool Success Rate:** Ratio of successful to total tool invocations.
- **Session Duration:** Total elapsed time in milliseconds.

Supporting Metrics (Tier 2) — Measures of Iteration and Coordination:

- **Total Rounds:** Count of tool execution rounds (batches of tools executed per request).
- **Retry Count:** Total tool invocation retries across all rounds.
- **Tool Retry Rate:** Ratio of retries to rounds; indicates iterative problem-solving frequency.
- **Invocations Per Request:** Average tools per request; reflects task complexity and thoroughness.
- **Unique Tools Used:** Count of distinct tool types invoked.

Agentic Activity (Tier 3) — Measures of Code and File Interaction:

- **File Modifications:** Count of file creation, deletion, or edit operations.
- **Read Operations:** Total file read operations.
- **Write Operations:** Total file write operations.
- **Read/Write Ratio:** Ratio of reads to writes; higher values suggest deliberate exploration before modification.

1.5.3 Functional Metrics (Both Studies)

To address **RQ3**, the final code artifacts (`.patch` files) were evaluated against a blind test suite:

- **Deliverable Completeness:** Presence of all required deliverables.
- **Functional Correctness:** Success/Failure on an automated E2E test harness.
- **Robustness:** Handling of edge cases (e.g., authentication failure, empty responses, etc.).

2 Results

2.1 Economic Performance (Study A)

[Table comparing Token/Cost metrics across APM, Spec-kit, and Plan Mode.] [Analysis: Does APM's overhead result in a linear or exponential cost increase? Is the cost justified by the output?]

2.2 Process Efficiency (Study B)

[Graphs showing User vs. Agent turns.] [Analysis: Did APM users intervene less? Did the Manager agent effectively absorb the coordination burden?]

2.3 Code Quality and Functional Success

[Pass/Fail rates for the automated E2E test suite.] [Qualitative analysis of the code: Did the agents successfully identify the AJAX endpoint?]

3 Discussion

3.1 The Cost of Context vs. The Cost of Failure

[Discuss the trade-off. APM likely has higher upfront costs (Setup Phase), but does it prevent the costly "Agentic Handoff Failures" observed in the baselines? How does the cost of the APM framework compare to the cost of the baselines?]

3.2 The Hybrid Model Efficacy

[Evaluate the strategy used in Study B. Did switching to a budget model for implementation degrade code quality, or did the strong plan from the Premium Manager protect against it? How does the hybrid model strategy compare to the single-agent or double-agent strategies in terms of code quality and robustness?]

3.3 Threats to Validity

- **Learning Effects:** Addressed via counterbalancing (Tool A/Tool B order).
- **Model Variance:** Mitigated in Study A by standardizing on Composer 1. Addressed in Study B by using a Claude Sonnet 4 for the planning phase of all tools, and a budget model for the implementation phase of all tools.
- **Participant Experience:** Variance in student skill levels (mitigated by the "Detailed PRD" which acts as a leveling mechanism and the "Default Prompt" containing hints and guidance for the agents).